

Vortex Tech Internship – Week 3



Basic Security Audit of a Sample Website

Name: Fatima Noor

Track: Cyber Security

Introduction

In this task, I performed a basic security audit of OWASP Juice Shop, a vulnerable website made for security practice. I set it up locally on Kali Linux using Docker and explored different features of the website. I tested user input and the login process to look for common security issues. I also used browser Developer Tools to inspect website requests and responses.

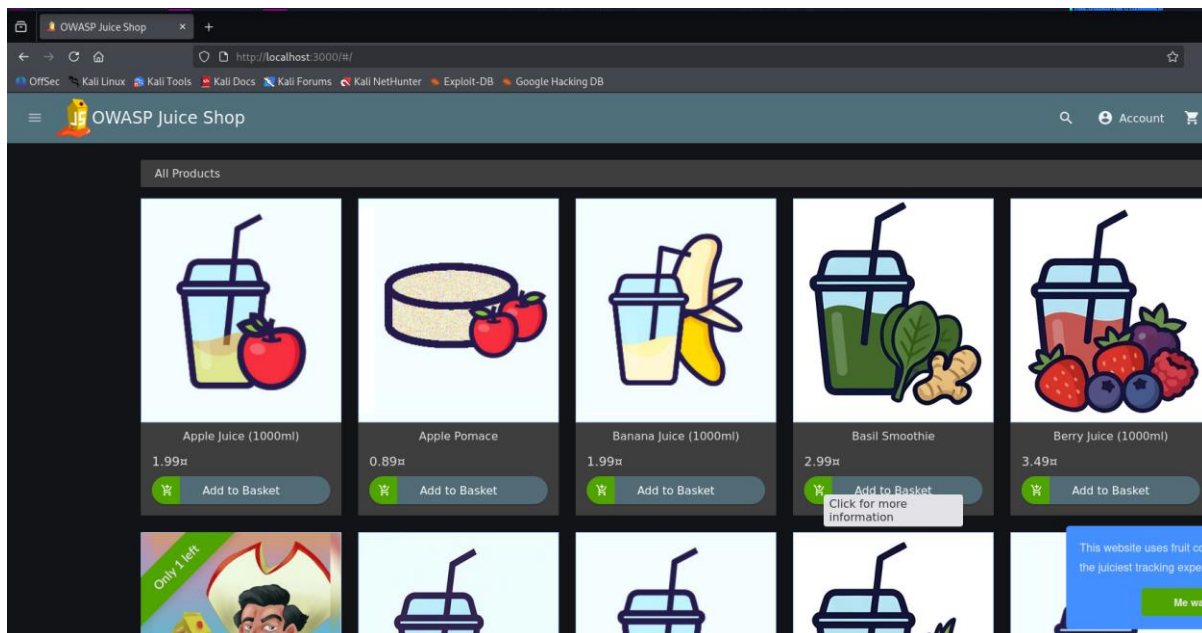
Juice Shop Setup

Docker was used to download and run the OWASP Juice Shop image. After the container was started, the application was accessed through localhost:3000

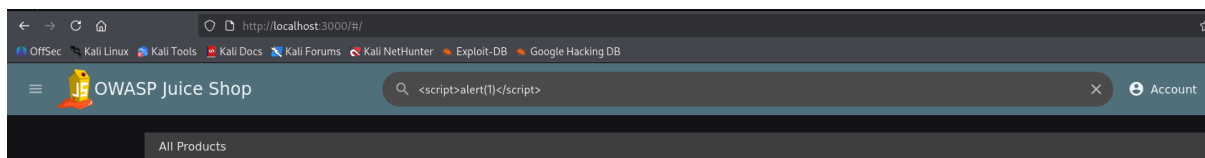
```
(fatima@kali)-[~]
└─$ sudo docker pull bkimminich/juice-shop
Using default tag: latest
latest: Pulling from bkimminich/juice-shop
1eed391ea893: Pull complete
c172f21841df: Pull complete
99515e7b4d35: Pull complete
99ba982a9142: Pull complete
d6b1b89eccac: Pull complete
2780920e5dbf: Pull complete
7c12895b777b: Pull complete
3214acf345c0: Pull complete
52630fc75a18: Pull complete
dd64bf2dd177: Pull complete
b839dfae01f6: Pull complete
ebddc55facdc: Pull complete
bdfd7f7e5bf6: Pull complete
2d4d7adf6272: Pull complete
cf397222899a: Pull complete
bd8962e29291: Pull complete
cac2ae0193cb: Pull complete
dd0d9bfd3bee: Pull complete
7cc70dfd88cf: Pull complete
8224d91da70b: Pull complete
1a4be5562d92: Pull complete
2ba730cb9f70: Pull complete
37c0829f1adc: Pull complete
467afe997cab: Pull complete
Digest: sha256:73c53fbf442e8337b3ea3d98c7e8550308854701ebdfce4cc39768f36b75430e
Status: Downloaded newer image for bkimminich/juice-shop:latest
docker.io/bkimminich/juice-shop:latest

(fatima@kali)-[~]
└─$ sudo docker run -d -p 3000:3000 bkimminich/juice-shop
c12fbdd1782ad63fe85a73131d999152801061a65215ce97f40b1f2d57286fa8
```

Juice shop homepage:

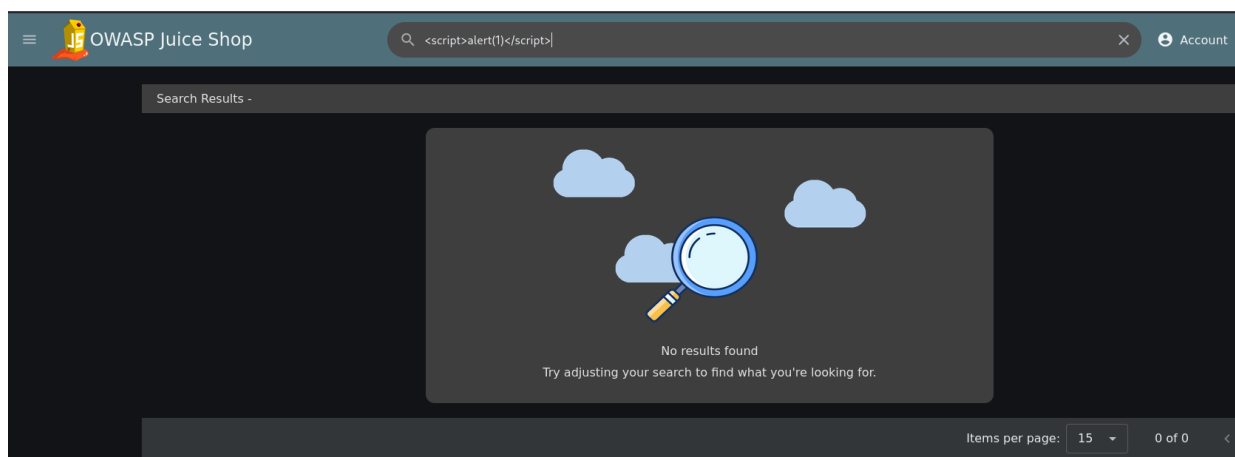


Finding #1: Cross-Site Scripting (XSS)



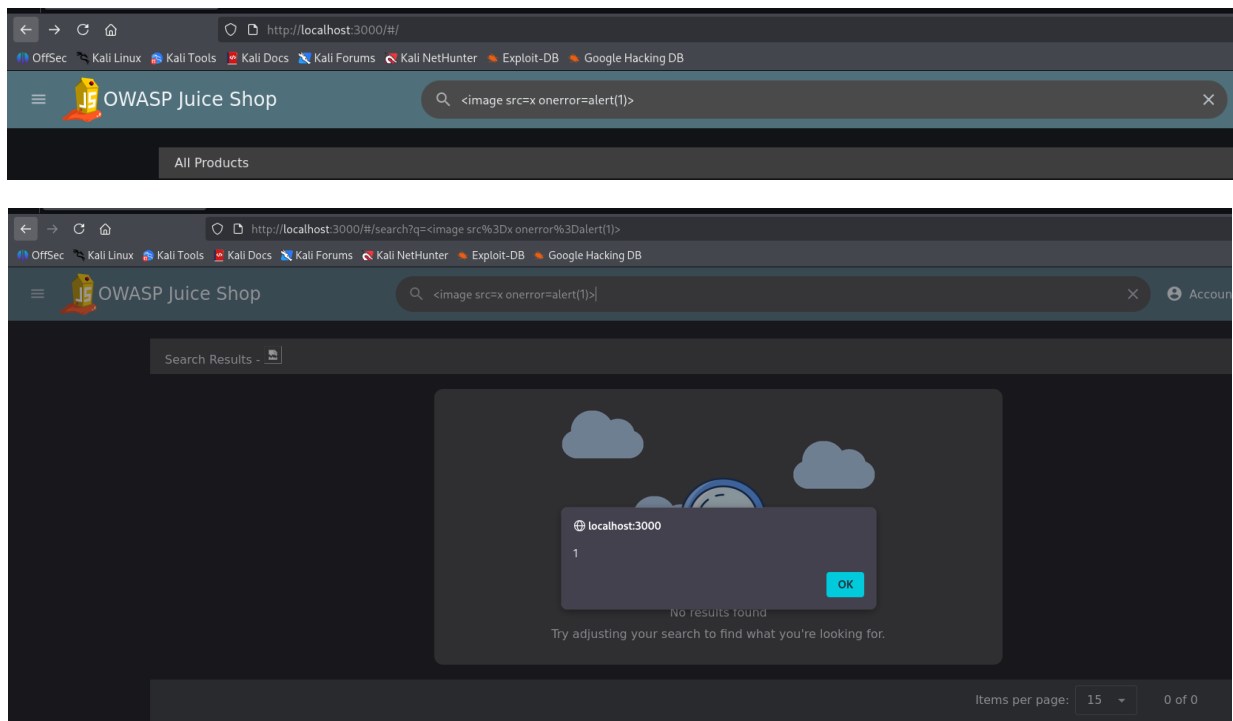
The payload(<script>alert(1)</script>) was entered into the search field.

The application did not execute it as JavaScript through that search result.



Then Entered this payload: `<img src=x onerror=alert(1)>`

JavaScript alert “1” appeared, confirming XSS.



Vulnerability Category: Cross-Site Scripting (XSS)

Finding

The second test payload worked and the website executed it as JavaScript. This shows that the search function is not handling the input safely.

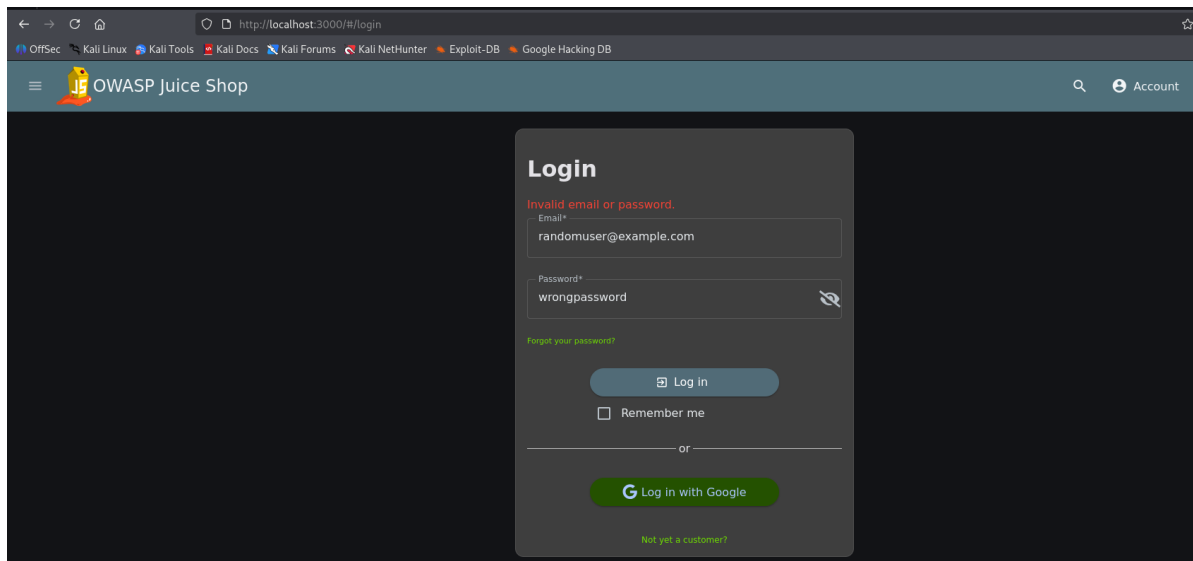
Potential Impact

XSS can allow an attacker to run JavaScript in another user's browser. This could be used to change the webpage or perform actions without the user's permission.

Remediation

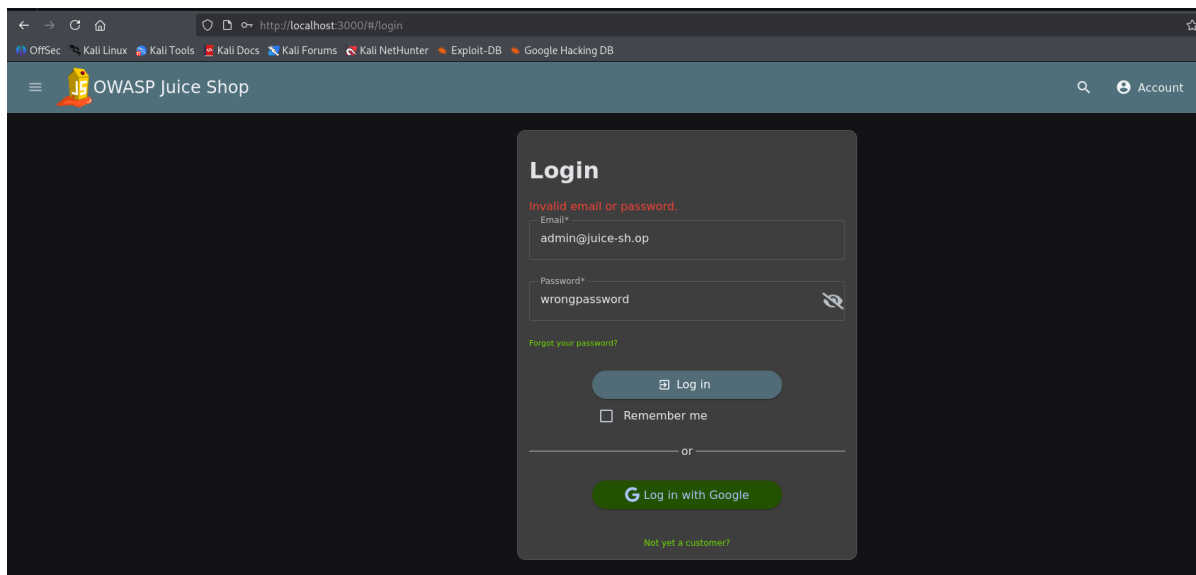
The application should properly check and clean user input before displaying it. User input should also be safely encoded so that scripts are treated as text instead of being executed.

Login / Authentication Test



Entered a random email (randomuser@example.com) with password (wrongpassword).

“Invalid email or password” was displayed.

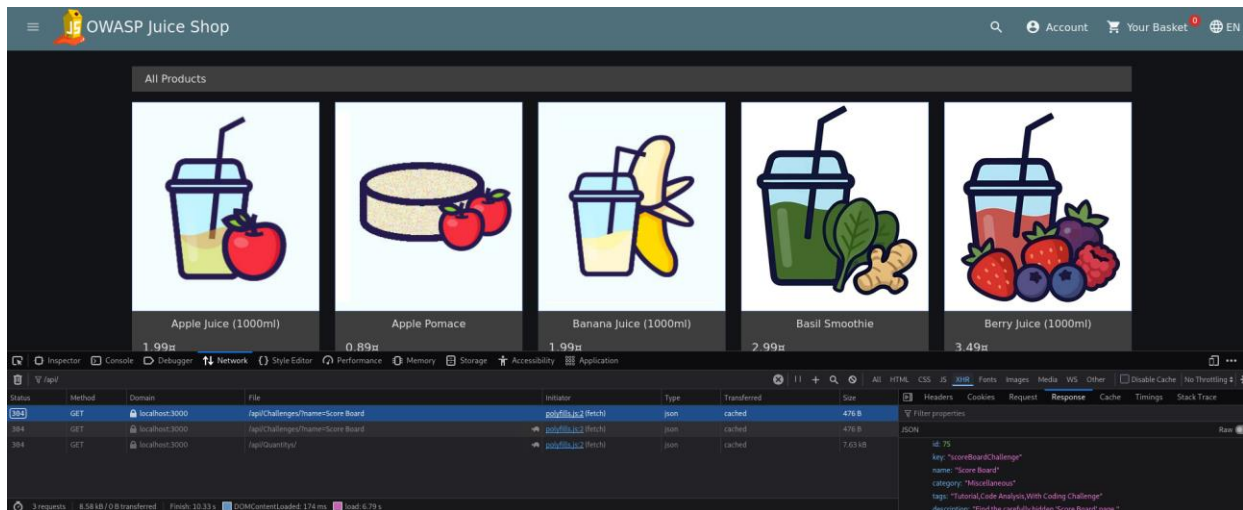


After finding (admin@juice-sh.op) in the Juice Shop review section, entered it with (wrong password).

“Invalid email or password” was displayed again.

Both inputs produced the same error message, so no information-revealing difference was observed during this test.

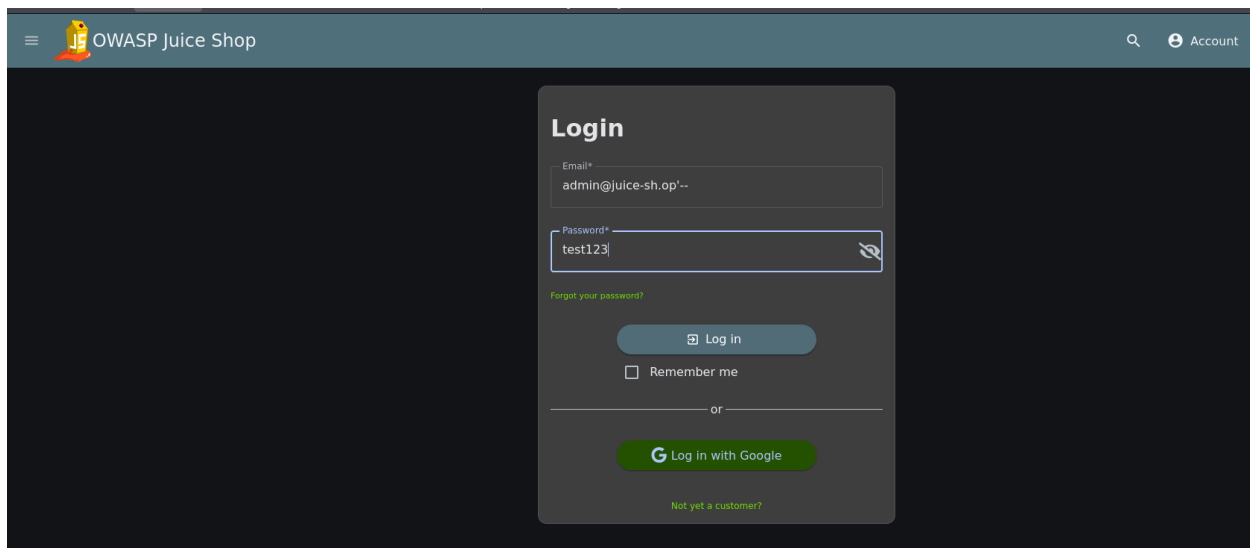
Developer Tools (Network/API Inspection)



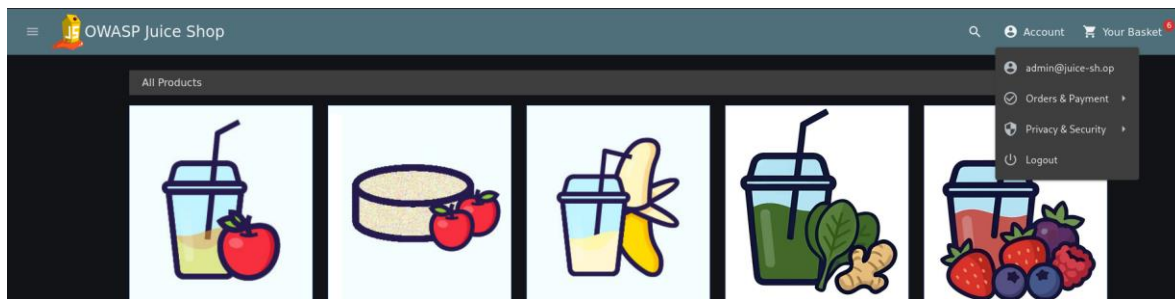
API responses were inspected using the browser's DevTools Network tab. The `/api/Challenges/?name=Score Board` response exposes challenge metadata such as the challenge ID, name, category, tags, and description. This information is not itself sensitive, but the same technique can be used to identify API responses that may expose sensitive information to the client.

Finding #2: SQL Injection / Authentication Bypass

Entered `admin@juice-sh.op'--` as the email with `test123` as the password.



Successfully logged in as administrator.



Vulnerability Category: SQL Injection (SQLi)

Finding

The login form accepted specially crafted input and allowed authentication without the correct administrator password.

Potential Impact

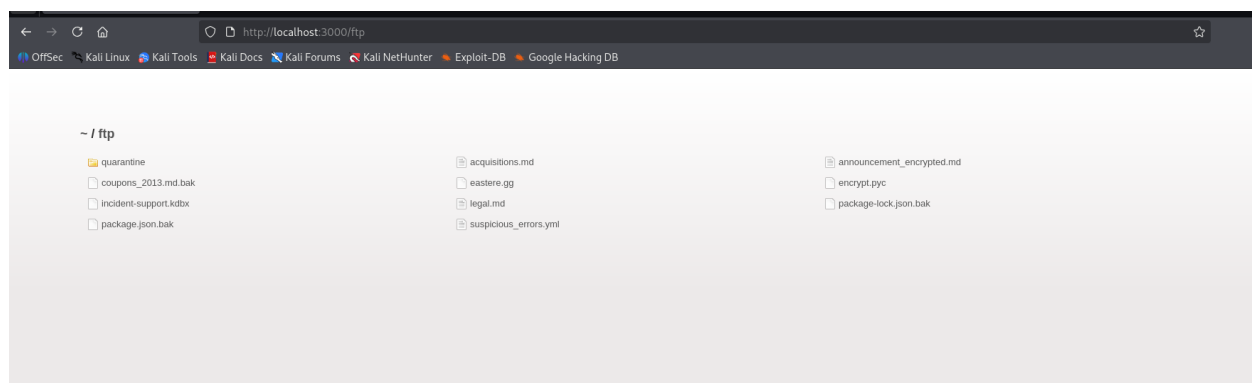
An attacker could potentially bypass authentication and gain unauthorized access to an account.

Remediation

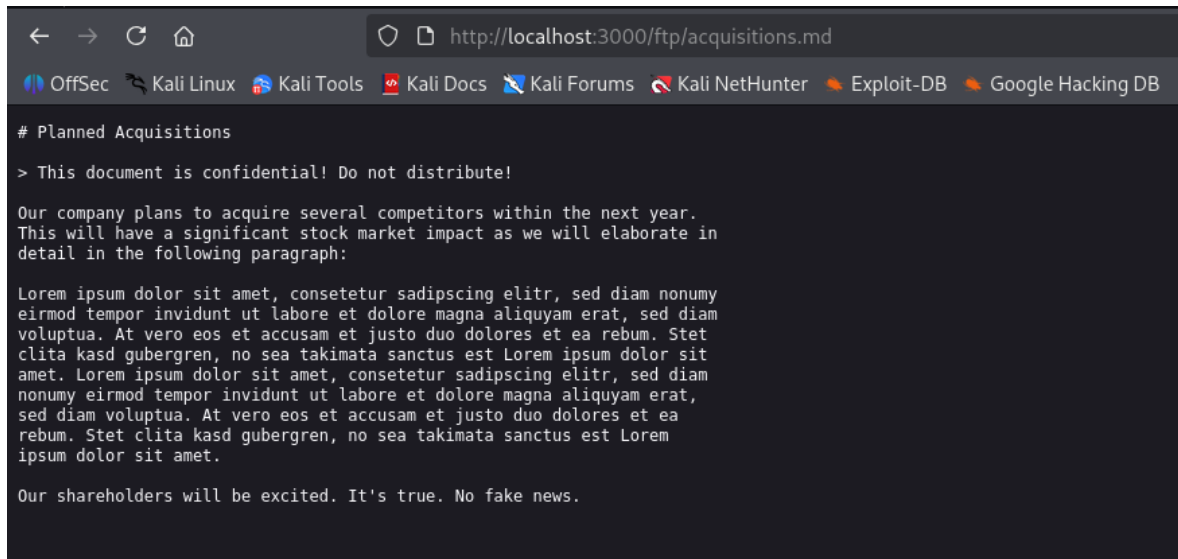
Use parameterized queries/prepared statements for database operations and validate user input to prevent SQL injection.

Finding #3: Sensitive Information Disclosure via Exposed FTP Directory

The /ftp directory was directly accessible through the web application. It exposed multiple files.



This file(acquisitions.md) contained information explicitly presented as confidential company acquisition information.



```
# Planned Acquisitions

> This document is confidential! Do not distribute!

Our company plans to acquire several competitors within the next year.
This will have a significant stock market impact as we will elaborate in
detail in the following paragraph:

Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam nonumy
eirmod tempor invidunt ut labore et dolore magna aliquyam erat, sed diam
voluptua. At vero eos et accusam et justo duo dolores et ea rebum. Stet
clita kasd gubergren, no sea takimata sanctus est Lorem ipsum dolor sit
amet. Lorem ipsum dolor sit amet, consetetur sadipscing elitr, sed diam
nonumy eirmod tempor invidunt ut labore et dolore magna aliquyam erat,
sed diam voluptua. At vero eos et accusam et justo duo dolores et ea
rebum. Stet clita kasd gubergren, no sea takimata sanctus est Lorem
ipsum dolor sit amet.

Our shareholders will be excited. It's true. No fake news.
```

Vulnerability Category: Sensitive Information Disclosure / Exposed Sensitive Files

Finding

The /ftp directory was directly accessible through the web application, exposing files including acquisitions.md, which contains confidential company acquisition information.

Potential Impact

Unauthorized users could access confidential business information that should not be publicly exposed.

Remediation

Restrict access to sensitive directories and files, remove confidential files from publicly accessible locations, and enforce proper authorization controls.

Findings Summary

Finding	Category	Evidence	Status
1	Cross-Site Scripting (XSS)	JavaScript alert triggered by search input	Confirmed
2	SQL Injection / Authentication Bypass	Authentication succeeded with crafted email input	Confirmed
3	Sensitive Information Disclosure	Publicly accessible /ftp directory and confidential file	Confirmed

Reflection

This task helped me understand how to find common security problems by testing a website step by step. The XSS test showed me why user input needs to be handled carefully. The SQL injection test helped me see how a poorly protected login system can be bypassed. I also learned that sensitive files can be exposed when proper access controls are not used. Using Developer Tools gave me a better understanding of how website requests and responses can be checked. Overall, this task gave me useful hands-on experience with basic web security testing and reporting.